gdb cheatsheet

Starting GDB

Command	Description
`gdb <program>`	Start GDB.
`gdb <program> [core dump]`	Start GDB with an optional core dump.
`gdb --args <program> <args...>`	Start GDB and pass arguments.
`gdb --pid <pid>`	Start GDB and attach to a process.
`set args <args...>`	Set arguments for the program to be debugged.
`show args`	Show current arguments.
`run`	Run the program.
`start`	Run and stop at `main`.
`kill`	Kill the running program.
`quit`	Exit GDB.

Breakpoints

Command	Description
`break <where>`	Set a new breakpoint.
`break file.cpp:42`	Break at a source line.
`break function_name`	Break at function entry.
`break Class::Method`	Break at class member function entry.
`break *0x00001234`	Break at a code address.
`delete <breakpoint#>`	Remove a breakpoint.
`clear`	Delete all breakpoints.
`clear file.cpp:42`	Delete breakpoints at a line.
`enable <breakpoint#>`	Enable a disabled breakpoint.
`disable <breakpoint#>`	Disable a breakpoint.
`info breakpoints`	Show breakpoints and watchpoints.
`set breakpoint pending on`	Allow unresolved breakpoints for future libraries.

Watchpoints and conditions

Command	Description
`watch <where>`	Set a new watchpoint.
`watch variable_name`	Break on write.
`rwatch variable_name`	Break on read.
`awatch variable_name`	Break on read or write.
`break/watch <where> if <condition>`	Conditional breakpoint or watchpoint.
`condition <breakpoint#> <condition>`	Set or change the condition of an existing breakpoint.
`ignore <id> 5`	Ignore the next five hits.
`commands <id> ... end`	Run commands when a breakpoint hits.

Stack and stepping

Command	Description
`backtrace` / `where`	Show the call stack.
`backtrace full` / `where full`	Show call stack with local variables.
`frame <frame#>`	Select a stack frame.
`up` / `down`	Navigate stack frames.
`step`	Go to the next source line, diving into functions.
`next`	Go to the next source line without diving into functions.
`finish`	Continue until the current function returns.
`continue`	Continue normal execution.

Variables and memory

Command	Description
`print/format <what>`	Print variable, memory, or register content.
`print variable`	Print a variable value.
`print *ptr`	Dereference a pointer.
`print *arr@N`	Print N array entries starting at `arr`.
`display/format <what>`	Print after each step or stop.
`display variable`	Auto-print at each breakpoint or stop.
`undisplay <display#>`	Remove a display entry.

Command	Description
`enable display <display#>` / `disable display <display#>`	Toggle display entries.
`x/nfu <address>`	Print memory.
`x/[COUNT][SIZE][FMT] <address>`	General memory inspection syntax.
`info proc mappings`	Show process memory layout.

x modifiers

Part	Meaning
Count	How many units to print; comes first if used.
Size	`b` byte, `h` half-word, `w` word, `g` giant word.
Format	`a`, `c`, `d`, `f`, `o`, `s`, `t`, `u`, `x`, `i` for pointer, char, signed, float, octal, string, binary, unsigned, hex, instruction.

Useful `<what>` forms

Form	Meaning
`expression`	Almost any C expression; function calls may need an explicit cast.
`file_name::variable_name`	Static variable from a file scope.
`function::variable_name`	Variable from a function scope, if available.
`{type}address`	Interpret memory at address as a given C type.
`$register`	Register content such as `$esp`, `$ebp`, or `$eip`.

Sources and TUI

Command	Description
`directory <directory>`	Add a source directory.
`show directories`	Show configured source directories.
`list`	Show current source context.
`list <filename>:<function>`	Show source around a function.
`list <filename>:<line_number>`	Show source around a line.
`set listsize <count>`	Set the number of source lines shown by `list`.
`show listsize`	Show the current `list` line count.
`gdb --tui <program>`	Start with text UI enabled.

Command	Description
`tui enable`	Enable TUI from inside GDB.
`layout src`, `layout asm`, `layout regs`	Select common TUI layouts.
`layout prev` / `layout next`	Cycle layouts.
`tui focus <layout>`	Focus a TUI pane.

Information commands

Command	Description
`info args`	Show current frame arguments.
`info locals`	Show current frame locals.
`info display`	Show display expressions.
`info sharedlibrary`	List shared libraries.
`info signals`	List signals and their behavior.
`info threads`	List threads.
`whatis variable_name`	Show variable type.
`ptype MyClass`	Show class or struct details.
`disassemble` / `disassemble <where>`	Disassemble current or selected location.

Signals and manipulation

Command	Description
`handle <signal> <options>`	Configure whether a signal prints, stops, or passes.
`handle SIGSEGV stop print`	Stop and print on SIGSEGV.
`handle SIGPIPE nostop noprint pass`	Pass SIGPIPE to the program.
`handle SIGUSR1 nopass`	Catch SIGUSR1 and do not pass it on.
`signal SIGUSR1`	Send a signal to the inferior.
`set var <name>=<value>`	Change a variable value.
`set variable x = 10`	Alternative variable assignment form.
`set $rax = 0`	Change a register value.
`return <expression>`	Force current function to return immediately.
`jump file.cpp:100`	Jump execution to another source line.
`call function(args)`	Call a function manually from GDB.

Threads, tracepoints, reverse, Valgrind

Command	Description
`thread <thread#>`	Choose a thread to operate on.
`thread apply all <command>`	Execute a command across all threads.
`break file.cpp:42 thread 3`	Break only in thread 3.
`trace function_name` / `trace file.c:42`	Create a tracepoint.
`actions` / `collect $regs,$locals,myvar` / `end`	Define tracepoint data collection.
`passcount 100 1`	Stop after collecting 100 hits.
`tstart`, `tstop`, `tfind`, `tdump`, `tstatus`, `tsave`, `target tfile`	Trace collection and analysis workflow.
`record full`	Enable recording for reverse debugging.
`reverse-step`, `reverse-next`, `reverse-continue`	Move backward through execution.
`valgrind --vgdb=yes --vgdb-error=0 --vgdb-stop-at=startup <app>`	Start Valgrind in GDB-connectable mode.
`target remote	vgdb`	
`monitor leak_check` / `monitor v.info location <addr>`	Query Valgrind state from GDB.